

1.2 Modularizacion

Cuando nuestros programas se vuelven más complejos, mantener todo el código en un solo archivo se vuelve ineficiente y difícil de manejar. La modularización es el proceso de dividir un programa grande en partes más pequeñas, lógicas e interrelacionadas, llamadas módulos. Cada módulo se encarga de una tarea específica, lo que facilita el desarrollo, la depuración y la reutilización del código.

Python tiene un sistema muy potente para modularizar, basado en dos conceptos clave: módulos y paquetes.

Módulos: Organizando nuestro Código

Un módulo en Python no es más que un archivo con extensión *.py*. Este archivo puede contener variables, funciones y clases que puedes usar en otros archivos de tu proyecto.

Ventajas de usar módulos:

- **Organización**: Agrupa el código relacionado en un solo lugar.
- **Reutilización**: Puedes usar el mismo módulo en diferentes proyectos sin necesidad de copiar y pegar el código.
- **Mantenibilidad**: Es más fácil corregir o mejorar un programa si está dividido en partes lógicas.
- **Evita colisiones de nombres**: Las funciones o variables de un módulo no interferirán con las de otro, ya que cada uno tiene su propio "espacio de nombres" (namespace). Es decir, cada función o variable tiene su Scope particular en el archivo en el cual estan definidos.

Creando un módulo simple:

Imaginemos que estamos creando un conjunto de funciones matemáticas básicas. En lugar de ponerlas en nuestro archivo principal, creamos un archivo llamado *operaciones.py*.

operaciones.py

```python
"""
Módulo de operaciones matemáticas básicas.
Este módulo contiene funciones para sumar, restar,
multiplicar y dividir.
"""

PI = 3.14159

def sumar(a, b):
    """Retorna la suma de dos números."""
    return a + b

def restar(a, b):
    """Retorna la resta de dos números."""
    return a - b

def multiplicar(a, b):
    """Retorna la multiplicación de dos números."""
    return a * b
```

Ver operaciones.py

Quiero aclarar que en Python no existe una palabra reservada *export* similar a la de JavaScript. El sistema de módulos de Python es diferente y más implícito. Cualquier función, clase o variable que se defina en un archivo Python (*.py*) se considera automáticamente parte del módulo y está disponible para ser importada por otros archivos.

Uso de *import*: Cómo Utilizar Módulos

Para poder usar las funciones y variables de nuestro módulo *operaciones.py* en otro archivo (por ejemplo, *main.py*), necesitamos importarlo. Existen varias formas de hacerlo.

Supongamos que *main.py* está en la misma carpeta que *operaciones.py*.

Forma 1: *import nombre_modulo*

Esta es la forma más común y recomendada. Importa todo el contenido del módulo, pero para usarlo, debes prefijar el nombre del módulo.

main.py

Python

```python
# Importamos el módulo completo
import operaciones

# Para usar sus funciones, usamos la sintaxis: modulo.funcion()
resultado_suma = operaciones.sumar(10, 5)
resultado_resta = operaciones.restar(10, 5)

print(f"La suma es: {resultado_suma}")
print(f"La resta es: {resultado_resta}")

# También podemos acceder a las variables del módulo
print(f"El valor de PI es: {operaciones.PI}")
```

Ver importacionSimple.py

Ventaja: Es muy claro de dónde viene cada función (*operaciones.sumar*), evitando confusiones.

Forma 2: *from nombre_modulo import elemento*

Si solo necesitas una o varias funciones específicas del módulo, puedes importarlas directamente.

main.py

Python

```python
# Importamos solo las funciones sumar y restar
from operaciones import sumar, restar

# Ahora podemos usarlas directamente sin el prefijo
resultado_suma = sumar(15, 7)
resultado_resta = restar(15, 7)

print(f"La suma es: {resultado_suma}")
print(f"La resta es: {resultado_resta}")

# Ojo: la variable PI no fue importada, por lo que esto daría un error:
# print(PI) # NameError: name 'PI' is not defined
```

Ver importacionEspecifica.py

Ventaja: El código es un poco más corto.

Desventaja: Si importas muchas funciones de diferentes módulos, podrías perder la pista de qué función pertenece a qué módulo.

Forma 3: Usando un alias (apodo) con *as*

Si el nombre de un módulo es muy largo o conflictivo, puedes asignarle un alias más corto.

main.py

Python

```python
# Importamos el módulo operaciones con el alias "ops"
import operaciones as ops

resultado = ops.multiplicar(8, 4)
print(f"La multiplicación es: {resultado}")
print(f"El valor de PI sigue siendo: {ops.PI}")
```

Ver importacionAlias.py

Ventaja: Muy útil para módulos con nombres largos como *matplotlib.pyplot as plt* o *pandas as pd*, una práctica estándar en la comunidad de ciencia de datos.

Forma 4: *from nombre_modulo import* * (No recomendada)

Esta sintaxis importa todo el contenido de un módulo al espacio de nombres actual, permitiéndote usar las funciones directamente.

main.py

Python

```
# Importamos todo desde operaciones
from operaciones import *

resultado = sumar(5, 5) # Se usa directamente
print(f"PI es {PI}")
```

Ver importaciónTotal.py

¿Por qué no se recomienda?

- "Contamina" el espacio de nombres: Llena tu archivo con un montón de variables y funciones que no sabes de dónde vienen.
- Puede causar colisiones: Si tu archivo main.py ya tiene una función llamada sumar, esta será sobrescrita por la que importaste, causando errores difíciles de rastrear. Es mejor ser explícito.

Paquetes: Organizando Módulos (**Codigo en carpeta)**

Cuando un proyecto crece mucho, puedes tener tantos módulos que hasta la carpeta principal se desordena. Un paquete es simplemente una carpeta que contiene varios módulos relacionados y un archivo especial llamado *__init__.py*.

Estructura de un paquete

Imaginemos que queremos crear una biblioteca más grande llamada *matematicas*.

```
mi_proyecto/
├── main.py
└── matematicas/
    ├── __init__.py
    ├── basicas/
    │   ├── __init__.py
    │   └── operaciones.py
    └── avanzadas/
        ├── __init__.py
        └── trigonometria.py
```

- *matematicas*: Es el paquete principal (una carpeta).
- *__init__.py*: Es un archivo (puede estar vacío) que le dice a Python: "Esta carpeta es un paquete". También se puede usar para inicializar código del paquete.
- *basicas* y *avanzadas*: Son sub-paquetes.
- *operaciones.py* y *trigonometria.py*: Son módulos dentro de los paquetes.

¿Cómo usamos los módulos dentro de un paquete?

Se utiliza la "notación de punto" (*paquete.modulo*).

main.py

Python

```python
# Importar el módulo 'operaciones' del sub-paquete 'basicas'
from matematicas.basicas import operaciones

# Importar una función específica del módulo 'trigonometria'
from matematicas.avanzadas.trigonometria import seno

# Usamos la función del módulo 'operaciones'
suma = operaciones.sumar(20, 10)
print(f"Suma desde el paquete: {suma}")

# Usamos la función importada directamente
resultado_seno = seno(90) # Suponiendo que la función 'seno' existe en trigonometr
print(f"Seno de 90: {resultado_seno}")
```

Creación de Bibliotecas Simples (**Codigo en carpeta)**

Una "biblioteca" es, en esencia, un paquete (o un conjunto de paquetes y módulos) diseñado para ser distribuido y reutilizado por otros desarrolladores. El paquete *matematicas* que diseñamos arriba es una biblioteca simple.

Pasos para crear tu propia biblioteca simple

- **Define el propósito**: ¿Qué problema resolverá tu biblioteca? (Ej: "Cálculos matemáticos").
- **Organiza la estructura de carpetas**: Crea una carpeta principal (el paquete) y dentro de ella, los módulos *.py* que necesites. No olvides el archivo *__init__.py* en cada carpeta que deba ser tratada como un paquete.
- **Escribe tu código**: Desarrolla las funciones, clases y variables dentro de tus módulos. Recuerda documentar tu código usando docstrings (las comillas triples *"""..."""*) para explicar qué hace cada función.
- **Prueba tu biblioteca localmente**: Desde un archivo fuera de la carpeta de tu biblioteca (como *main.py* en el ejemplo), importa tus módulos y prueba que todo funcione como esperas.

Ejemplo

Imagina que tu proyecto tiene la siguiente estructura:

```
guia_programacion/
├── mi_app.py
└── mi_biblioteca/
    ├── __init__.py
    └── utilidades.py
```

mi_biblioteca/utilidades.py

Python

```python
"""
Módulo de utilidades generales.
"""
def saludar(nombre):
    """Genera un saludo personalizado."""
    return f"¡Hola, {nombre}! Bienvenido a la modularización."

def es_par(numero):
    """Verifica si un número es par."""
    return numero % 2 == 0
```

mi_app.py

Python

```python
# Importamos el módulo 'utilidades' de nuestro paquete 'mi_biblioteca'
from mi_biblioteca import utilidades

# Usamos las funciones
print(utilidades.saludar("Estudiante"))

numero_a_verificar = 10
if utilidades.es_par(numero_a_verificar):
    print(f"El número {numero_a_verificar} es par.")
else:
    print(f"El número {numero_a_verificar} es impar.")
```

Con estos conceptos, ya tienes una base sólida para entender y aplicar la modularización en tus proyectos de Python, creando código más limpio, organizado y profesional.



1.3 Entornos Virtuales y Gestión de Dependencias

Imagina que estás trabajando en dos proyectos diferentes:

- Proyecto A: Un sitio web que necesita la librería Django en su versión *3.2*.
- Proyecto B: Un script de análisis de datos que necesita Django en su versión más reciente, la *4.1*.

Si instalas *Django* en tu sistema de forma global, solo puedes tener una versión a la vez. Si actualizas a la *4.1* para el Proyecto B, ¡probablemente el Proyecto A deje de funcionar! Este problema se conoce como "conflicto de dependencias".

Un entorno virtual soluciona este problema creando una "burbuja" o un directorio aislado para cada proyecto. Dentro de esta burbuja, puedes instalar las librerías y versiones específicas que ese proyecto necesita, sin afectar a otros proyectos ni a la instalación global de Python en tu sistema.

En resumen, usar entornos virtuales te permite:

- **Evitar conflictos**: Cada proyecto tiene sus propias dependencias aisladas.
- **Mantener tu sistema limpio**: Las librerías se instalan por proyecto, no globalmente.
- **Facilitar la colaboración**: Puedes compartir una lista de las dependencias exactas de tu proyecto, permitiendo que otros lo repliquen fácilmente.

Herramientas Clave: *venv* y *pip*

- *venv*: Es el módulo estándar de Python (incluido desde Python 3.3) para crear entornos virtuales.
- *pip*: Es el gestor de paquetes de Python. Lo usamos para instalar, actualizar y eliminar librerías (dependencias) dentro de nuestro entorno virtual activado.

Flujo de Trabajo con *venv* y *pip* (El Método Estándar)

Este es el proceso paso a paso para gestionar un proyecto de Python de manera profesional.

Paso 1: Crear el Entorno Virtual

1. Abre una terminal o línea de comandos.
2. Navega hasta la carpeta raíz de tu proyecto. Si no existe, créala (*mkdir mi_proyecto* y luego *cd mi_proyecto*).
3. Ejecuta el siguiente comando:

Bash

```
python -m venv venv
```

Desglose del comando:

- *python -m venv*: Le dice a Python que ejecute el módulo *venv*.
- *venv*: Es el nombre que le daremos a la carpeta que contendrá nuestro entorno virtual. Usar "venv" es una convención muy extendida y recomendada.

Después de ejecutar el comando, verás una nueva carpeta llamada *venv* dentro de tu proyecto. Esta carpeta contiene una copia de Python y todo lo necesario para instalar librerías de forma aislada.

¡Importante! Siempre debes agregar el nombre de esta carpeta (e.g., *venv/)* a tu archivo *.gitignore* para evitar subir el entorno virtual a tu repositorio de código.

Paso 2: Activar el Entorno Virtual

Crear el entorno no es suficiente; necesitas "activarlo" para empezar a usarlo. La activación modifica tu sesión de terminal para que use el Python y el *pip* de la carpeta *venv*.

- En Windows (cmd.exe):

Bash

```
venv\Scripts\activate
```

- En Windows (PowerShell):

PowerShell

```
.\venv\Scripts\Activate.ps1
# Si hay un error, puede que necesites cambiar la política de ejecución:
# Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
```

Sabrás que el entorno está activo porque el nombre del entorno (*venv*) aparecerá al principio de la línea de comandos.

“(venv) C:\Users\TuUsuario\mi_proyecto>”

Cualquier librería que instales con *pip* a partir de ahora, se instalará únicamente dentro de esta "burbuja".

Paso 3: Instalar y Gestionar Librerías con *pip*

Con el entorno activo, puedes usar *pip* para manejar las dependencias de tu proyecto.

- Instalar una librería:

```Bash
pip install requests
```

Esto instalará la última versión de la popular librería *requests* para hacer peticiones HTTP.

- Instalar una versión específica:

```Bash
pip install django==3.2.12
```

- Listar las librerías instaladas:

El comando *pip list* te mostrará lo que hay instalado en el entorno, pero *pip freeze* es más útil para el siguiente paso.

```Bash
pip freeze
```

La salida será algo como:

```
certifi==2022.9.24
charset-normalizer==2.1.1
idna==3.4
requests==2.28.1
urllib3==1.26.12
```

Paso 4: Crear el Archivo *requirements.txt*

¿Cómo le dices a un compañero de equipo (o a tu "yo" del futuro) qué librerías necesita instalar para que el proyecto funcione? La respuesta es un archivo *requirements.txt*.

Este archivo es una simple lista de las dependencias del proyecto. Para generarlo automáticamente, usa *pip freeze* y redirige la salida a un archivo:

```Bash
pip freeze > requirements.txt
```

Ahora tendrás un archivo *requirements.txt* en tu proyecto con el contenido que vimos antes. Este archivo sí debes subirlo a tu repositorio de Git.

Paso 5: Recrear el Entorno desde *requirements.txt*

Si otro desarrollador (o tú mismo en otra máquina) clona tu proyecto, no necesitará adivinar las dependencias. Solo tendrá que seguir estos pasos:

1. Crear y activar un entorno virtual (Pasos 1 y 2).
2. Instalar todas las dependencias listadas en el archivo con un solo comando:

```Bash
pip install -r requirements.txt
```

pip leerá el archivo e instalará todas las librerías con sus versiones exactas. ¡Esto garantiza que todos trabajen con el mismo entorno!

Paso 6: Desactivar el Entorno Virtual

Cuando termines de trabajar en tu proyecto, puedes salir de la "burbuja" con un simple comando:

```Bash
deactivate
```

El prefijo (*venv*) desaparecerá de tu terminal, indicando que has vuelto a la configuración global de tu sistema.

Resumen del Ciclo de Vida de un Entorno Virtual

1. Crear: *python -m venv venv* (una vez por proyecto).
2. Activar: *source venv/bin/activate* o *venv\Scripts\activate* (cada vez que vayas a trabajar en el proyecto).
3. Trabajar:

- Instalar librerías con *pip install <libreria>.*
- Actualizar tu *requirements.txt* con *pip freeze > requirements.txt*.

4. Desactivar: *deactivate* (cuando termines).

Dominar este flujo de trabajo es un paso esencial para pasar de escribir scripts simples a desarrollar aplicaciones de Python robustas y colaborativas.



1.4 Manejo de Errores

Hasta ahora, es probable que hayas visto tu programa detenerse bruscamente y mostrar un mensaje de error en la consola (un "Traceback"). Esto ocurre cuando Python encuentra una situación que no puede resolver, como intentar dividir por cero o acceder a un archivo que no existe.

Un programa que se cierra inesperadamente frente a un usuario final es una mala experiencia. El manejo de errores es el conjunto de técnicas que nos permiten anticipar estos posibles fallos, capturarlos y responder de una manera controlada y elegante, en lugar de dejar que el programa "crashee".

Excepciones

En Python, los errores que ocurren durante la ejecución de un programa se llaman excepciones. Cuando se produce un error, Python "lanza" (raises) un objeto de excepción. Si este objeto no es "atrapado" (caught), el programa se detiene.

Piensa en una excepción como una alarma de emergencia. Nuestro trabajo como programadores es instalar "extintores" en el código para manejar esas alarmas.

Ejemplos de excepciones comunes:

- *ValueError*: Ocurre cuando una función recibe un argumento del tipo correcto pero con un valor inapropiado.

Python

```
# Esto lanza un ValueError porque "hola" no se puede convertir a número.
numero = int("hola")
```

- *ZeroDivisionError*: La clásica excepción al intentar dividir cualquier número por cero.

Python

```
# Esto lanza un ZeroDivisionError.
resultado = 10 / 0
```

- *FileNotFoundError*: Se produce cuando intentamos abrir un archivo que no existe en la ruta especificada.

Python

```
# Esto lanza FileNotFoundError si el archivo no existe.
archivo = open("archivo_inexistente.txt", "r")
```

- TypeError: Ocurre al intentar realizar una operación en un tipo de dato incorrecto.

Python

```
# Esto lanza un TypeError porque no se puede sumar un número y un texto.
suma = 5 + "gatos"
```

El Bloque *try* y *except*: Atrapando Errores

La herramienta principal para manejar excepciones es el bloque *try...except*.

- *try*: En este bloque colocas el código que podría causar una excepción.
- *except*: En este bloque colocas el código que se ejecutará solo si ocurre una excepción en el bloque try.

Sintaxis

Python

```
try:
    # Código propenso a errores
    ...
except TipoDeExcepcion:
    # Código a ejecutar si ocurre esa excepción
    ...
```

Ejemplo

Python

```python
try:
    numerador = int(input("Introduce el numerador: "))
    denominador = int(input("Introduce el denominador: "))
    resultado = numerador / denominador
    print(f"El resultado es: {resultado}")

except ZeroDivisionError:
    print("Error: No se puede dividir por cero.")

except ValueError:
    print("Error: Debes introducir números válidos.")

print("El programa continúa después del error.")
```

Análisis del ejemplo

1. El código dentro de *try* intenta convertir las entradas a enteros y realizar la división.
2. Si el usuario introduce *0* como denominador, se lanza un *ZeroDivisionError*. Python salta al bloque *except ZeroDivisionError* y ejecuta el *print* correspondiente.
3. Si el usuario introduce texto (ej: "cinco"), la función *int()* lanza un *ValueError*. Python salta al bloque *except ValueError*.
4. Crucialmente, el programa no se detiene. La línea "El programa continúa..." se ejecuta en ambos casos de error.

Siempre es mejor ser específico con el tipo de excepción que esperas (*except ZeroDivisionError*:) en lugar de usar un *except*: genérico. Esto te permite reaccionar de forma diferente a cada tipo de error.

El Bloque *finally*: Código que Siempre se Ejecuta

Python

```python
try:
    # Código propenso a errores (ej: trabajar con un archivo)
    ...
except:
    # Manejar el error
    ...
finally:
    # Este código se ejecuta SIEMPRE
    ...
```

Ejemplos

Python

```python
archivo = None # Inicializamos la variable fuera del try
try:
    archivo = open("mi_documento.txt", "w")
    archivo.write("Hola, Programación 2!")
    # Forzamos un error para ver qué pasa
    resultado = 10 / 0

except FileNotFoundError:
    print("Error: El archivo o directorio no existe.")

except Exception as e: # 'Exception' es una clase base que atrapa casi cualquier e
    print(f"Ocurrió un error inesperado: {e}")

finally:
    print("Ejecutando el bloque finally.")
    if archivo:
        archivo.close()
        print("El archivo ha sido cerrado correctamente.")
```

Ver TryExceptFinally.py

Importante, como dice el comentario, la clase *Exception* nos permite atrapar cualquier error de manera genérica, usar con suspicacia.

En este ejemplo, aunque 10 / 0 causa una excepción, el bloque *finally* se ejecuta de todos modos, asegurando que el archivo se cierre y no quede corrupto o bloqueado. Sin embargo es bueno presentar mas ejemplos del *finally*, como liberar recursos de red o base de datos;

Python Copiar

```python
try:
    conexion = conectar_bd()
    resultado = conexion.ejecutar("SELECT * FROM usuarios")
except Exception as e:
    print("Error en la consulta:", e)
finally:
    conexion.cerrar()  # Se cierra la conexión pase lo que pase
```

Mostrar mensajes finales o limpiar la interfaz

Python Copiar

```python
try:
    print("Procesando datos...")
    procesar()
except Exception:
    print("Hubo un error.")
finally:
    print("Proceso terminado.")  # Siempre se muestra
```

La Sentencia *raise*: Lanzando Nuestras Propias Excepciones

No solo podemos atrapar excepciones, también podemos lanzarlas nosotros mismos. La palabra clave *raise* nos permite crear y lanzar una excepción de forma deliberada.

¿Por qué haríamos esto? Para validar condiciones que no son errores para Python, pero sí son errores para la lógica de nuestro programa.

Ejemplo: Validando la edad de un usuario

Python

```python
def registrar_usuario(edad):
    """Registra un usuario si la edad es válida."""
    if edad < 0:
        # La edad negativa no es un error para Python, pero sí para nuestra lógica
        # Por lo tanto, lanzamos una excepción para señalarlo.
        raise ValueError("La edad no puede ser un número negativo.")
    elif edad < 18:
        print("Usuario registrado, pero es menor de edad.")
    else:
        print("Usuario mayor de edad registrado correctamente.")

# --- Ahora usamos la función dentro de un bloque try ---
try:
    registrar_usuario(25)   # Funciona
    registrar_usuario(15)   # Funciona
    registrar_usuario(-5)   # Esto lanzará nuestro error

except ValueError as e:
    # Atrapamos la excepción que nosotros mismos lanzamos
    print(f"Error en el registro: {e}")
```

1. La función *registrar_usuario* tiene una regla de negocio: la edad no puede ser negativa.
2. Si la edad es menor que cero, usamos *raise ValueError(...)* para lanzar un error con un mensaje descriptivo.
3. Cuando llamamos a *registrar_usuario(-5),* la excepción se lanza y es capturada por nuestro bloque *except ValueError*, permitiéndonos manejar el error de forma controlada.

En definitivas cuentas

- *Excepción*: Un error que ocurre durante la ejecución del programa.
- *try*: Define un bloque de código donde pueden ocurrir excepciones.
- *except*: Define un bloque que maneja una excepción específica si ocurre en el try.
- *finally*: Define un bloque que se ejecuta siempre, haya ocurrido una excepción o no. Ideal para tareas de limpieza.
- *raise*: Permite forzar el lanzamiento de una excepción, útil para validar la lógica de tu aplicación.



1.6 Control de Versiones con Git y GitHub: El Superpoder del Programador

Imagina que estás trabajando en tu proyecto final. Guardas tu archivo como *proyecto.py*. Haces algunos cambios y lo guardas como *proyecto_v2.py*. Luego, te das cuenta de un error y creas *proyecto_v2_corregido.py*. Unos días después, tienes una carpeta llena de archivos como *proyecto_final_AHORA_SI.py* y no tienes idea de cuál es la versión correcta o qué cambios hiciste en cada una.

El control de versiones resuelve este caos. Es un sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo lo largo del tiempo, permitiéndote volver a versiones específicas más adelante.

¿Qué es Git?

Git es un sistema de control de versiones distribuido. Piénsalo como un "álbum de fotos" de tu proyecto. Cada vez que haces un cambio importante, tomas una "foto" (llamada *commit*) de todo tu proyecto en ese momento. Esto te permite:

- Ver un historial completo de todos los cambios.
- Saber quién hizo qué cambio y cuándo.
- Volver a una versión anterior si algo sale mal.
- Trabajar en nuevas funcionalidades de forma segura sin afectar la versión principal estable.

Git funciona en tu computadora local. Es la herramienta que gestiona el historial de tu código.

¿Qué es Github?

GitHub es una plataforma web que utiliza Git. Es como un "Google Drive" o una red social para tus proyectos de código. Te permite:

- Alojar tus repositorios (proyectos) en la nube, creando una copia de seguridad centralizada.
- Colaborar con otros desarrolladores de manera sencilla y organizada.
- Mostrar tu trabajo al mundo, creando un portafolio profesional.
- Participar en proyectos de código abierto (open source).

En resumen: Git es la herramienta que usas en tu terminal para gestionar el historial de cambios. GitHub es el servicio online donde guardas y compartes tus proyectos gestionados por Git.

Flujo de Trabajo Básico: De tu PC a GitHub

Vamos a realizar nuestra primera interacción con Git. El proceso puede parecer intimidante al principio, pero el 80% del tiempo usarás solo un puñado de comandos.

Paso 0: Configuración Inicial

Antes de empezar, debes decirle a Git quién eres. Abre tu terminal o consola y escribe:

Bash

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu_email@ejemplo.com"
```

Paso 1: Crear un Repositorio

Un repositorio (o "repo") es simplemente una carpeta que contiene tu proyecto. Tienes dos formas de empezar:

A. Iniciar un nuevo proyecto localmente.

1. Crea una nueva carpeta para tu proyecto y navega hacia ella en tu terminal.
2. Ejecuta el comando:

Bash

```
git init
```

B. Clonar un proyecto existente desde GitHub

1. Ve a GitHub y encuentra un repositorio que quieras copiar (puede ser uno tuyo o de otra persona).
2. Haz clic en el botón verde "Code" y copia la URL (usualmente termina en *.git*).
3. En tu terminal, ejecuta:

Bash

```
git clone URL_DEL_REPOSITORIO
```

Esto descargará el proyecto completo con todo su historial a una nueva carpeta en tu computadora.

Paso 2: El Ciclo Fundamental (Añadir, Confirmar, Empujar)

Este es el proceso que repetirás una y otra vez mientras trabajas.

1. Revisa el estado (*git status)*

Antes de hacer nada, es una buena práctica ver el estado de tu proyecto. Este comando te dice qué archivos han sido modificados, cuáles son nuevos y no están siendo rastreados, etc.

Bash

```
git status
```

2. Añade los cambios al "Área de Preparación" (*git add*)

Git no guarda automáticamente cada cambio. Tú tienes que decirle qué cambios específicos quieres incluir en la próxima "foto" (*commit*). A esto se le llama añadir archivos al Staging Area (Área de Preparación).

Bash

```
# Para añadir un archivo específico
git add nombre_del_archivo.py

# O, para añadir todos los archivos modificados y nuevos (muy común)
git add .
```

3. Confirma los cambios (*git commit*)

Ahora que tus cambios están en el área de preparación, es hora de tomar la "foto". Cada commit debe ir acompañado de un mensaje descriptivo que explique qué cambiaste.

Bash

```
git commit -m "Agregué la función de saludo al usuario"
```

¡Listo! Has guardado una nueva versión en el historial de tu computadora local.

4. Empuja los cambios a GitHub (*git push*)

Hasta ahora, todos los cambios viven solo en tu PC. Para subirlos a GitHub y que otros puedan verlos (o para tener tu copia de seguridad), necesitas "empujar" tus commits.

Bash

```
git push origin main
```

- *origin* es el nombre por defecto del repositorio remoto (tu repo en GitHub).
- *main* es el nombre de la rama principal de tu proyecto.

Paso 3: Mantenerse Sincronizado (*git pull*)

Si estás trabajando con otras personas, o si hiciste cambios desde otra computadora, necesitarás actualizar tu repositorio local con los cambios que están en GitHub.

Bash

```
git pull origin main
```

Este comando descarga los commits más recientes del repositorio remoto y los fusiona con tu versión local.

Como tarea

Con ayuda de la documentación, github y algún modelo de lenguaje si lo deseas, crea un repositorio en tu github llamado “Lenguajes de programación II - 05S-2603-D1”, y sube el archivo actual en el que has anotado, probado y experimentado en clase llamado “Laboratorio 1 – [FECHA ACTUAL]”